

Class Inheritance

Class: blueprint for creating objects. Classes create reference types. Creating object gets reference to memory address, not object itself, All statements should be wrapped either in a class or a method. Attributes (instance variables/field) describe an object's state, Method: behaviour objects can perform, **Constructor:** special methods to **initialise objects (variable+value)**. Valid Constructor: same name as class, **NO return type (not even void)**, properly initialises variables, calls super() if extending abstract/parent class, implements abstract parent's abstract method. **In Java, a method signature consists of: the method name + the parameter list (types and order)**. It **does NOT include the return type** and it does NOT include the throws clause.

'Static' belong to class (**one shared copy**). Static methods cannot use 'this' because they are not associated with any specific object and can only access static members. Instance (non-static) methods implicitly receive the this reference, giving them access to both instance variables and static variables, and allowing them to represent behaviours tied to particular objects. Static methods should **be accessed via the class name**. Instance methods cannot be called from static methods unless done **through an object reference such as new A().method()**. Final variables are immutable(a constant), final methods cannot be overridden, final classes cannot be extended.

Inheritance: class inherits attributes and methods from superclass/parent."is-a" relationship using 'extends'. **public class Subclass extends Superclass** super(...) calls parent constructor, this(...) calls same class, other constructor. Both **must be the first statement**, but cannot appear together. Calling parent methods by super() or super.method(). **If superclass has no default constructor, subclass must call super(args)**.

Access control (broadest → narrowest): public (IA everywhere) → protected (IA in subclasses) → default (IA only same package) → private (not IA). Inherited, accessible: IA

Polymorphism means that a parent-type reference can point to a child object, and **method calls** are always determined by **the object's actual runtime type**. Fields/static are not polymorphic and are always resolved at compile time based on the **reference type**. Upcasting from a subclass to a superclass happens automatically and is always safe, while downcasting requires an explicit cast and should be checked with **instanceof** to avoid ClassCastException. Polymorphic Arrays: Store different object types in one array using parent type

UML: + ~ -# ~: public / private / protected / package-private
Italic text = abstract(class or method names). Underlined text = static: Indicates static fields or static methods. bold text: inherited stuff from others. Methods with no return value must end with void
Hollow triangle arrow = inheritance (extends); Dashed line + hollow triangle = interface implementation (implements), start with <<interface>>; Plain arrow = unidirectional association (A holds a reference to B)

Java packages correspond directly to the **folder structure**. Every source file must declare a package that matches its directory location. An **import** statement tells Java where to find a class, and a fully qualified name uses the full path in the form **package.ClassName**. Classes in the **default package cannot be imported by classes in named packages**. If classes are not located under the same root directory, the classpath must be set manually during compilation and execution. The protected modifier is visible to subclasses across packages but not to classes in other packages. The default modifier is only visible within the same package. A class in a different package cannot be instantiated directly without an import or using its fully qualified name. Package declarations and directory structure must be consistent.

JAR(Java Archive)=Compressed collection of .class files, Preserves package structure, Can be executed. Creating a JAR: jar -cfm project.jar manifest.mf path/*.class
Order MUST be: jar file name, manifest file, class files
Running a JAR: java -jar project.jar
Manifest file (manifest.mf): Manifest must specify Main-Class for JAR to run Main-Class: JarMain
If missing manifest, error: no main manifest attribute
Using JAR as a library via classpath:
javac -cp ./home/project.jar JarUse.java
Run with JAR: java -cp ./home/project.jar JarUse
Meaning: "Look inside this JAR when finding classes."

Enums

Enum: data type to define a variable as a set of fixed, predefined constants. Used for known, unchanging collection of values. **Type Safety:** Any defined type variables can only be from predefined values, prevents compile-time errors by preventing assignment of arbitrary integer or string. **Readability:** if (light == TrafficLight.GREEN)//better than vague-meaning number. **Fixed Instances:** each constant listed in an enum like TrafficLight.RED is a **public, static and final instance** of that enum type. "Singletons" created when the class is loaded, because only one instance of each, **== used for comparison**. All enums inherit from the java.lang.Enum class. **values()**: returns array with all enum constants, **values()[index]** gets the enum constant at position index; **valueOf(String name)**: returns enum constant that matches given name; **name()**: returns constant name as a string (e.g., "RED"); **ordinal()**: returns constant's index.
enum can have fields (variables), can define a constructor to initialise the fields. **Constructors must be private (or package-private)**, cannot directly instantiate an enum with new, cannot create instance from outside, can define regular methods with behaviour for enum instances, can implement interfaces(public enum Suits implements Iterable<String>). Declaring **abstract method inside enum** forces each enum constant (object) to define its own version of the behavior.

Text File I/O

Text file I/O: Scanner is used to read input from keyboard:new Scanner(System.in); files,or strings.Always call close(). **For file reading:** import java.io.;
File file = new File("data.txt");
Scanner sc = new Scanner(file); loop with hasNextLine() / nextLine().
For writing:PrintWriter writer = new PrintWriter(new File("out.txt")); ...writer.println(...); ...writer.close();
Handle File I/O exception with **try-catch** or use **try-with-resources to auto-close**.
try (**Scanner scan = new Scanner(new File(filename))**) { **File f = new File("output.txt");**
while (scan.hasNextLine()) { **try** {
//use hasNext...() before next...(); **PrintWriter writer = new**
PrintWriter(f);
System.out.println(scan.nextLine()); **writer.println("Line 1");**
writer.println(3.14);
writer.close();
} catch (FileNotFoundException e) {
System.out.println("Cannot
create file!");
e.printStackTrace(); **}**
scan.nextLine(); // skip existing 'n' int age = Integer.parseInt(scan.nextLine());
System.out.print("Enter name: ");
String name = scan.nextLine();
Throw raises an exception, throws declares it.
next():Reads the next token(single word). Return a String.**nextLine():**Reads an entire line including spaces. Return a String.**nextInt():**Reads the next integer value. Return int.**nextDouble():**Reads the next decimal (floating-point) value. Return double.**hasNext():**Checks if there are more tokens to read. Return boolean. **hasNextInt()** – Checks if the next token is an integer. Return boolean.

Some complements:
initialize array:
int[] numbers = new int[16];
int[] nums1 = {1, 2, 3, 4};
int[] nums2 = new int[] {1, 2, 3, 4};
int[][] matrix = new int[3][3];
int[] store the value (**primitive type**); String[] store **reference**;
In reference type objects are originally assigned to null
Integer.parseInt(arr[i]); convert String element to an int, Integer is a wrapper class an object
++n(post-increment)/++n(pre-increment)
System.out.println(n++ + ++n); // n=3,res = 8
public static void main(String[] args)
must name your file <classname>.java, Compile program using **javac Hello.java**, Run the program by entering **java Hello**. **Java 11**: execute single-file programs using source file, without compiling them. syntax: java Hello.java
Comparing Strings: use equals, not ==; Use .toArrayArray() to iterate over string, **charAt**

Collections (java.util.*)

A List type is an abstract data type

Map, List, Set override toString(), arrays don't, so System.out.println() prints their contents in bracket format ([...], [...]).Arrays do NOT override toString(), so printing an array prints memory address-like strings. Map is NOT a subtype of Collection.
Unchecked operations occur when the compiler cannot verify type safety, usually because **raw types** are used, like creating an ArrayList without specifying type:
Map provides a mapping of an object to location where that element is stored. note that you have to use **wrapper classes** like Integer inside the <>.
HashMap has key-value pairs. **HashSet is unordered, stores unique elements only**.methods: **put(K key, V value)** adds or updates an entry; **get(K key)** retrieves the value associated with a given key, always returns value type because collections only store objects; **remove(K key)** removes an entry by key; **containsKey(K key)** checks if a key exists; **containsValue(V value)** checks if a value exists; **keySet()** returns a set of all keys; **values()** returns a collection of all values; **size()** returns the number of entries; and **clear()** removes all entries from the map.map.getOrDefault(key, defaultValue) returns the value associated with the key if it exists in the map; or null

LinkedList chains elements and their values, can add/remove from both ends. Keep elements in **arbitrary positions of memory, special functions addFirst() and addLast(). ArrayList**- resizable array, **auto-resizes, ordered list with duplicates**, can store various types of objects(**cannot store primitive types due to type erasure**)
ArrayList<String> list = new ArrayList<String>(>);, instantiate array by List<Integer> nums = Arrays.asList(10, 20, 30);
add(E element) adds element to end, (list.add("Hi")), **add(int index, E element)** inserts element at index (list.add(0, "First")), **get(int index)** retrieves element at index (list.get(2)), **set(int index, E element)** replaces element at index (list.set(1, "New")), **remove(int index)** removes element at index (list.remove(0)); **remove(Object o)** removes element by value (list.remove("Hi")), **size()** returns number of elements (list.size()), **clear()** removes all elements (list.clear()); **contains(Object o)** checks if element exists (list.contains("Hi")), **isEmpty()** checks if list empty (list.isEmpty()).**indexOf(x)** → returns index of x, or -1 if not found.
Collections.sort(List<T> list);static method from java.util.Collections, sort a list **in ascending orderFrom(Comparable)**
In Java 8, List also has a default instance method(uses provided Comparator): **sort(Comparator<? super T> c):** nums.sort((a, b) → b - a); **// sort descending** **List<Map.Entry<String, Integer>> sortedList = new ArrayList<>(wordCount.entrySet());** **sortedList.sort((a, b) -> b.getValue() - a.getValue());**

Interfaces / Abstract Classes

Abstract class can implement interface,they both cannot be instantiated, but interface variables can reference any implementing object, which allows polymorphism. Dynamic binding ensures that the correct method runs based on the actual object type at runtime. Abstract classes may define partial behavior and hold state, while interfaces cannot (unless using **default** methods).
If subclass does not implement all abstract methods from parent, it must also be declared abstract. An abstract method must be overridden
Interfaces and abstract class both cannot be declared final. **Interface can extend multiple interfaces**, but class can extend ONE superclass.
Abstract classes may have constructors, called by the subclass's constructor using super(), and may **maintain internal state**. **Interfaces have no constructors** or instance variables except constants.
All methods in interface are implicitly **public&abstract**, all variables are **public,static,final**.Interface method cannot be protected / private. Java 8→interfaces may include default and static methods; private helper methods (Java 9)
static methods are called via **InterfaceName.method()**. If two interfaces define the same **default** method, the implementing class must **override it to resolve the conflict**. (if it's static, then it will not be inherited). interface A **extends B(interface → interface)**; class A **implements B(class → interface)**; class A **extends B(class → class)**
Abstract Class used when classes need shared concrete methods, for protected/private members, different access modifiers needed.
Interface used when defining contract/behavior, multiple classes with different implementations, multiple inheritance 'types'.
Superclass implementation always 'wins' over a default interface method.

Iterable represents an object that can be looped over, and Iterator performs the actual looping.
public interface Iterable<T> { Iterator<T> iterator(); }
Iterable provides an iterator() method that returns an Iterator. Any class that can be used in a for-each loop must implement Iterable.
Iterator<T> it = iterable.iterator();
while (it.hasNext()) { T e = it.next(); }
hasNext() tells whether there are more elements without advancing, next() returns the current element and moves the cursor forward; calling next() with no remaining elements must throw NoSuchElementException.
Comparable defines the natural ordering of a class. The class itself implements Comparable<T> and provides the **compareTo(T other)** method. compareTo returns **a negative number, zero, or a positive number** to indicate whether this object is less than, equal to, or greater than the other. **players.sort((p1, p2) → Integer.compare(p2.goalsScored, p1.goalsScored))**;
This gives every object a built-in default ordering. Use Comparable when the type has one obvious, natural way to be sorted.
A Comparator<T> is a separate object that implements **compare(T a, T b)**. It allows you to create multiple different sorting strategies without modifying the class being sorted. Use Comparator when you want custom or alternative orderings.

Recursion

Recursion:a method calls itself to solve a problem. Each recursive call works on a smaller subproblem until reach a base case.Components:**Base Case:** Where recursion stops;**Recursive Case:** Where method calls itself with modified input;**Progress:** Each call must move towards base case
int factorialIterative(int n) {
int result = 1;
for (int i = 1; i <= n; i++) { result *= i; }
return result; }
int factorialRecursive(int n) {
if (n == 1) return 1;
return n * factorialRecursive(n - 1); }
Memoization:a technique to **store the results** of expensive function calls and reuse them when the same inputs occur.mainly used for Repeated Calculations. a "cache" for your recursive function.
public class FibonacciMemoized {
private Map<Integer, Integer> cache = new HashMap<>();
public int fibonacci(int n) {
// Check cache first
if (cache.containsKey(n)) {return cache.get(n);}
// Base cases
if (n == 0) return 0;
if (n == 1) return 1;
int result = fibonacci(n - 1) + fibonacci(n - 2);
cache.put(n, result);
return result;
}
}
// fibonacci(40) now takes milliseconds instead of seconds!→efficiency
We use a HashMap because if the result is in the cache, we return it immediately; if not, we continue the recursive computation and then store the newly computed value in the map.

Generics

handle multiple different types,no rewrite the same code. generics are invariant.

```
// Simple generic class
class Box<T> {
    private T item;
    public void setItem(T item) { this.item = item; }
    public T getItem() { return item; }
}

// More complex example
public class LinkedListNode<T extends Comparable<T>> {
    private T item;
    public T findMax() {
        if (next == null) { return value; }
        T maxInRest = next.findMax();
        return (value.compareTo(maxInRest) > 0) ? value : maxInRest;
    }
}

class Main {
    public static void main(String[] args) {
        Box<String> stringBox = new Box<>();
        stringBox.setItem("Hello");
        System.out.println(stringBox.getItem()); // Hello
    }
}
```

The extends keyword means **upper bound, even to interfaces**: class Cage<T extends Animal> {}

```
public class NumberBox<T extends Number> {...}
NumberBox<String> strBox = new NumberBox<>("Hi"); // error!
public static void addAll(List<? super Integer> list) {
    list.add(10); // allowed
    list.add(5);
}
```

E - Element (used in collections); K - Key; V - Value; N - Number; T - Type; S, U, V - 2nd, 3rd, 4th types

generic method: declares its own type parameter(s), independent of any class-level generics.

```
public static <T> void printArray(T[] array) {
    for (T element : array) { System.out.println(element); }
}
```

Overloading / Overriding

Overloading(Compile-time polymorphism):**same class**, same name, **different number/type/ order of parameters**, resolved at compile-time, can have different return types.

Overriding(Runtime polymorphism): **parent-child**, same signature (name+parameter list), resolved at runtime (dynamic binding), return type must be **covariant**, access modifier cannot be more restrictive. Cannot override private, static, or final methods.

Instance attribute cannot be overridden.

```
protected void foo(String a, int b); //parent class method
protected void foo(int b, String a); // public void foo(String a, String b) // allowed
protected void foo(String a, int c) // private int foo(String a, String b) // not allowed
```

JUnit Testing

Black Box Testing:Test functionality **not knowing internal code**. Based on **specifications**; checks what the **system** does. **White Box Testing**:Test **internal logic** and code paths. Tester knows the source code;to ensure all branches and conditions are executed.**Unit Testing**:Test the logic of smallest unit of code **in isolation** to confirm it behaves correctly. **Integration Testing**: Test the **interaction between multiple units** to ensure data flow and interfaces work correctly together. **Positive Case**: A test case uses valid data to check if the software behaves as intended.To confirm that the main features work correctly under expected conditions.**Negative Case**: A test case uses invalid data to see how the software responds,check the software's robustness and error handling. It should fail by showing an error message. **Edge Case**: A specific type of test case (positive or negative) targets the boundaries (null input, empty arrays, boundary values, out-of-range values)

JUnit:Test methods are marked with the **@Test** annotation so that JUnit can automatically detect and execute them. To control test execution flow, JUnit annotations: @BeforeClass and @AfterClass run once before and after all tests (must be static), **@Before** and **@After** run before and after each test case. **assertEquals** to compare values, **assertNotNull** to ensure inequality, **assertTrue** and **assertFalse** to test conditions, and **assertNull** or **assertNotNull** to verify null references. When an assertion fails, JUnit reports a failure

```
import static org.junit.Assert.*; //for asserts
import org.junit.*; //for annoations
assertEquals(expected, actual); assertTrue(condition); assertNull(obj); assertNotNull(obj);
General cases:Create normal, valid inputs and check expected output.
@Test
void testAddition() {
    int result = 2 + 3;
    Assertions.assertEquals(5, result, "Expected 5 as the result of 2 + 3");
}
Edge cases:
    • null inputs
@Test
void testItemNotFound() {
    String item = null; // Simulating a search result where the item wasn't found
    Assertions.assertNull(item, "Item should be null if not found");
    • Empty lists / strings
    • Minimum & maximum values
    • Boundary conditions (inside/outside allowed range)
assertThrows(IllegalArgumentException.class, () -> obj.method(null));
```

Type Erasure:At runtime, all generic type information is removed. List<Integer> and List<String> become the same raw type List. It ensures backward compatibility but also causes limitations like **can't use generic type checks with instanceof**, cannot create arrays of generic types because of type erasure. Use List<T> instead, **List<T> list = new ArrayList<>();** // Can do: **if (obj instanceof List<>){}** // OK

Static and Generics:Static fields and methods cannot use class-level generic parameters, because **generics belong to instances, not the class itself**.

```
class Box<T> {
    // GOOD - Use method-level type parameter
    public static <T> T createDefault() {return null;}
}

// Multiple Bounds: A generic type can have multiple bounds, but only one of them can be a class; the rest must be interfaces.
class Data<T extends Number & Comparable<T>> {}
Here T must be a subtype of Number and implement Comparable<T>.
Exception Restriction:Generic types cannot directly extend Throwable and create a generic exception class because type erasure removes type information needed for exception handling.
class MyException<T> extends Exception {} // compile error
```

Interface	Purpose	Required Methods
Comparable<T>	Make objects comparable, used for sorting	<code>compareTo(T other)</code>
Comparator<T>	External comparator, used for sorting	<code>compare(T a, T b)</code>
Iterable<T>	Supports for-each	<code>iterator()</code>
Iterator<T>	Iterates through a collection	<code>hasNext()</code> + <code>next()</code>
Collection<T> / List<T> / Set<T>	Collection interfaces	Many methods, e.g., <code>add</code> , <code>remove</code>

Anonymous Class / Lambda Expression

Anonymous class:a class with no name, a way to declare and instantiate a class at the same time in one expression. Create a small, one-time-use object that implements an interface or extends a class. **No Name**: It is defined and instantiated in a single expression using the **new keyword**; **One-Time Use**: It's designed to be used exactly where it is created; **Must Extend or Implement**: An anonymous class must **either Extend one existing class OR implement one existing interface**. The syntax **combines instantiation with a class definition**: **new SuperInterfaceOrClass() { // ... fields ... // ... method overrides ... }**; **No Constructor**: An anonymous class cannot have an explicit constructor because it has no name. If you need to run initialisation code, you can use an **instance initialiser block** (just a set of {...} brackets). **Scope Access**: An anonymous class can access members (fields and methods) of its enclosing class. It can also **access local variables** from the method where it is defined only if they are **final** or never changed after being initialised.

Can be stored and used in HashMap:
HashMap<String, IntegerBinaryOperation> operations = new HashMap<>();
operations.put("ADD", new IntegerBinaryOperation() { public int apply(int x, int y) { return x + y; } });
System.out.println(operations.get("ADD").apply(1, 1));

override method in an anonymous subclass: subclass's version replaces the parent class's version at runtime.

Lambda expression (Java 8):an anonymous function, a short, concise block of code that takes in parameters and returns a value. Like a method without a name, can pass it around as if it were an object or a variable. to implement **a functional interface (an interface with only one abstract method)**. In interface, **default method cannot be written as a Lambda expression** since Lambda expression can only appear as an object/value, not as the body of a method declaration, but λ expression can be used in a default method.

(parameters) $\rightarrow$ { body } **Parameters**: A list of parameters for the "method." (): No parameters; (x) or x: One parameter (parentheses are optional); (x, y): Multiple parameters. (usually don't need to declare the type, like (int x, int y), as the compiler can infer it.)**->**' separates the parameters from the body. **Body**: The code to be executed.**For a block of code, use curly braces**; For single expression,omit the braces and 'return' : return the value of the expression automatically.

```
HashMap<String, IntegerBinaryOperation> operations = new HashMap<>();
operations.put("ADD", (x, y) -> x + y); // key: "ADD"; value: (x, y) -> x + y
Lambda Examples:
// Lambda comparator: sort by length (short -> long)(always for sorting)
Collections.sort(names, (a, b) -> a.length() - b.length());
// Predicate: check if number is even (always boolean)
Predicate<Integer> isEven = n -> n % 2 == 0;
Collections.sort(list, new Comparator<Integer>() {
    // Filter using removeIf
    List<Integer> evens = new ArrayList<>(nums);
    evens.removeIf(isEven.negate()); // remove not-even
    // Consumer: print each element(always void, used for printing and saving)
    Consumer<String> printer = s -> System.out.println("Item: " + s);
    items.forEach(printer);
    Collections.sort(list, (a, b) -> b.compareTo(a));
}
Lambda parameter types can be omitted only when the compiler can infer them from the functional interface.
```

Exceptions

Error & Exception:Subclasses of the class **Throwable**. **Error(compile-time error)**: irrecoverable and usually not handled by the program.Causes: Missing ; at the end of a statement. Mismatched {} or () brackets. Using undeclared variables. Wrong type assignments(incompatible types)

Runtime Errors (java.lang.Error): The runtime Errors are always vital and **should not be handled** by the programmer (using try-catch blocks)A run-time error will only occur when the code is actually running. Causes: Infinite recursions/Out of memory, **StackOverflowError**

Checked Exceptions:exceptions need to be handled at **compile-time**.must be caught or declared in the method signature using throws. **IOException, FileNotFoundException, SQLException**.

Unchecked Exceptions/Runtime Exceptions:occur **at execution**; should be caught but do not have to be declared with throws.**ArithmeticException**; **ArrayIndexOutOfBoundsException**; **NullPointerException**

Logic Bugs by Programmers:wrong output. Causes:Wrong formula or algorithm. Incorrect loop boundaries (off-by-one errors). Using == instead of .equals() for string comparison. Misplaced conditions in if-else statements.

try: place your code that might throw an exception. **catch**: place your what to do if the exception happens.only runs if the try block throws a matching exception. **finally**: an optional block that **always executes after the try-catch block**. even runs if the try block throws an exception that isn't caught. **throw: manually create and throw an exception**. use when detect an error state in your own logic: if (age < 0) { throw new IllegalArgumentException(); } **throws: used in a method signature**. just pass the responsibility up the chain, forcing whatever method calls it to either use a try-catch block or declare throws. public void readFile(String fileName) throws FileNotFoundException {}

In Java, catch blocks must be ordered from the most specific exception (subclass) to the most general exception (superclass); otherwise, a compile-time error

```
catch(IndexOutOfBoundsException e)
catch(RuntimeException e)
catch(Exception e)
```

If a parent type (like Exception) comes first, it will catch all exceptions, making child catches unreachable $\rightarrow$ compilation error.

Custom Exception: Extend **Exception**: a Checked Exception. The compiler forces the calling method to handle this exception (using try-catch or throws). use for predictable, recoverable errors that the caller should know about. Extend **RuntimeException**: an Unchecked Exception. The compiler does not force the caller to handle it. use For programming errors or internal logic failures. The custom exception class should provide at least two standard constructors that call super() to set the (error) message: A no-argument constructor. A constructor that takes a String message.(Optional) A constructor that takes a String message and a Throwable cause (for exception chaining).

```
class MyException extends RuntimeException {
    public MyException(String message) { super(message); }
}

public class Main {
    public static void main(String[] args) { throw new MyException("My Exception!"); }
}
```

What you print	Output format
<code>System.out.println(list)</code>	Prints the <i>entire list structure</i> : [...] with commas and spaces
<code>System.out.println(list.get(i))</code>	Prints only the element itself , e.g., A
<code>System.out.println(itr.next())</code>	Prints only the element , e.g., A
<code>for (String s : list)</code>	Prints each element on its own line
<code>System.out.println(list.toString())</code>	Same as printing the list directly $\rightarrow$ [...]

Wildcards

super cannot be used in type parameter declaration; only extends. **? is completely unknown, so adding anything except null is unsafe**

wildcard:a placeholder ? for an **unknown type** in generics.Can have **bounds (upper or lower)**;mostly in method parameters, collections, or interfaces. **Unbounded wildcard ?** : List<?> list = new ArrayList<Integer>(); "a list of some unknown type." Read: You can only treat elements as **Object**. Write: You cannot **add anything (except null)**. list.add(10); //compile error Object o = list.get(0); // safe **Upper-bounded wildcard ? extends T** T's child class: List<? extends Number> nums = new ArrayList<Integer>(); "a list of some unknown subtype of Number." **Read: Safe to read as Number.** Write: Cannot add anything (except null). Number n = nums.get(0); //safe nums.add(5); //not allowed

PECS:Producer Extends (read/produce from it safely).

Lower-bounded wildcard ? super T T's parent class

```
List<? super Integer> list = new ArrayList<Number>();
"a list of some unknown supertype of Integer." Only safe read as Object. Can safely add Integer or its subclasses.
list.add(5); // safe
Object o = list.get(0); // only safe type to read from
Integer i = list.get(0); // compile error
PECS:Consumer Super (write/consume into it safely).
```

"extends $\rightarrow$ read only"
"super $\rightarrow$ write only"
• ? extends T $\rightarrow$ Producer $\rightarrow$ Extends $\rightarrow$ Read
• ? super T $\rightarrow$ Consumer $\rightarrow$ Super $\rightarrow$ Write

```
public static void copy(List<? super Number> src, List<? extends Number> dest) {
    for (Number n : src) {
        dest.add(n);
    }
}
```

src is read-only (? extends Number) and dest is write safe (? super Number). This is the canonical "produce extends, consumer super" example.